

Pitch to Postgres: Designing and Querying a Relational Database for FIFA World Cup Data

Milestone 3: Loading the data (03_load_data.ipynb)

DS604 Introduction to Data Management. Group: Devanshi Dudhatra (202618027), Pujita Sunapu (202618003), Rahul Saha (202618037)

Data: Joshua C. Fjelstul, *The Fjelstul World Cup Database*, <https://github.com/jfjelstul/worldcup>, licensed CC-BY-SA 4.0. **Modifications:** the tables are normalised as described in Milestone 2 (copied and derived columns are not loaded). Text markers become NULL, and the group names of 18 matches are aligned with `groups` (1982 second stage, 1950 final round). No data values are changed.

What this notebook does

1. Connects to PostgreSQL using a SQLAlchemy engine built **only from the** `.env` **file**, so no credentials appear in the code.
2. Creates the tables by running `sql/ddl.sql`.
3. Reads the 27 CSV files with `pd.read_csv`.
4. Cleans them explicitly, documenting each fix with the number of values it changed.
5. Loads them **in dependency order** with `DataFrame.to_sql(..., if_exists="append", index=False)` into the tables that already exist from the DDL.
6. Checks the row count of every table against its CSV and runs integrity checks.

Switching databases. Only the `.env` file decides where the data goes: the local test database now, the course server (`10.100.71.21`) later. No cell needs editing. This executed copy was run against the **local** test database.

0. Setup and connection

```
In [1]: import sys, time, inspect
from pathlib import Path

import pandas as pd
from sqlalchemy import text

ROOT = Path.cwd() if (Path.cwd() / "src").exists() else Path.cwd().parent # works
# from project root or notebooks/
sys.path.insert(0, str(ROOT / "src"))
import db # builds the engine from .env (src/db.py)
import clean # cleaning steps and column lists (src/clean.py)

pd.set_option("display.max_columns", 30)
pd.set_option("display.width", 200)
DATA = ROOT / "data"
```

```

engine = db.get_engine()
IS_LOCAL = db.load_settings()["DB_HOST"] in {"localhost", "127.0.0.1", ":::1"}
print("Target database:", db.describe())          # user@host:port/db only, never
the password
print("Local test database:", IS_LOCAL)

```

Target database: postgres@localhost:5432/worldcup schema: (default search_path)
Local test database: True

```

In [2]: with engine.connect() as conn:
        info = conn.execute(text("SELECT version(), current_database(), current_user,
current_schema()")).one()
        print("Server  :", info[0].split(",")[0])
        print("Database:", info[1], "| user:", info[2], "| schema:", info[3])

```

Server : PostgreSQL 18.3 on x86_64-windows
Database: worldcup | user: postgres | schema: public

1. Create the tables from sql/ddl.sql

The course rule is that `to_sql` must **append into tables that already exist**. The DDL therefore runs first, so the data takes on our types, keys and constraints rather than ones pandas would guess.

- **Local test database:** `sql/reset_local.sql` first drops the project tables, so the notebook can be rerun from scratch. This happens only when the host is local.
- **Course server:** nothing is dropped. `ddl.sql` uses `CREATE TABLE IF NOT EXISTS` and contains no DROP, TRUNCATE or DELETE.

```

In [3]: def run_sql_file(path):
        with engine.begin() as conn:          # one transaction: all or
nothing
            conn.exec_driver_sql(path.read_text(encoding="utf-8"))

        if IS_LOCAL:
            run_sql_file(ROOT / "sql" / "reset_local.sql")
            print("Local test database reset (project tables dropped).")
        else:
            print("Course server: no reset; tables are only created if they do not exist.")

        run_sql_file(ROOT / "sql" / "ddl.sql")

        tables = pd.read_sql_query("""
            SELECT table_name FROM information_schema.tables
            WHERE table_schema = current_schema() AND table_type = 'BASE TABLE' ORDER BY
table_name""", engine)
        print(f"{len(tables)} tables exist after running ddl.sql")
        print(", ".join(tables.table_name))

```

Local test database reset (project tables dropped).
28 tables exist after running ddl.sql
award_winners, awards, bookings, confederations, goals, group_standings, groups, host_countries, manager_appearances, manager_appointments, managers, matches, penalty_kicks, player_appearances, players, positions, qualified_teams, referee_appearances, referee_appointments, referees, squads, stadiums, substitutions, team_appearances, teams, tournament_stages, tournament_standings, tournaments

```

In [4]: # Safety check: appending into tables that already hold data would duplicate rows
        (and fail on the keys).
        counts = {t: pd.read_sql_query(f"SELECT count(*) AS n FROM {t}", engine).n[0] for t
in clean.LOAD_ORDER}

```

```

non_empty = {t: n for t, n in counts.items() if n}
if non_empty:
    raise RuntimeError(f"These tables already contain data, so loading stops here: {non_empty}. ")
    "Agree in the group before changing anything on the server.")
print("All 27 data tables are empty and ready for loading.")
print("positions lookup (seeded by ddl.sql):", pd.read_sql_query("SELECT count(*) AS n FROM positions", engine).n[0], "rows")

```

All 27 data tables are empty and ready for loading.
positions lookup (seeded by ddl.sql): 21 rows

2. Read the 27 CSV files with `pd.read_csv`

`keep_default_na=False` stops pandas from silently turning strings such as "NA" into missing values. Only truly empty cells become NaN, and the text markers "not available" and "not applicable" are handled explicitly in step 3.

```

In [5]: raw = {}
for name in clean.LOAD_ORDER:
    raw[name] = pd.read_csv(DATA / f"{name}.csv", keep_default_na=False, na_values=[
        ""], encoding="utf-8")

csv_summary = pd.DataFrame([(n, df.shape[0], df.shape[1]) for n, df in raw.items()],
                           columns=["table", "csv_rows", "csv_columns"])
print(csv_summary.to_string(index=False))
print("Total rows:", csv_summary.csv_rows.sum())

```

table	csv_rows	csv_columns
confederations	6	5
awards	8	5
stadiums	185	8
teams	84	11
players	7907	12
managers	357	6
referees	380	9
tournaments	21	18
tournament_stages	107	16
groups	117	7
qualified_teams	457	8
host_countries	22	7
tournament_standings	84	7
group_standings	458	19
squads	10142	12
manager_appointments	469	10
referee_appointments	511	10
matches	900	37
team_appearances	1800	36
referee_appearances	900	15
manager_appearances	1842	17
player_appearances	18623	22
goals	2548	27
bookings	2466	26
substitutions	6464	24
penalty_kicks	279	19
award_winners	132	12

Total rows: 57269

3. Cleaning

Each fix is a small, named function in `src/clean.py`, applied in the order below. The reasons come from the data profiling in Milestone 0 (`docs/dataset_understanding.md`).

#	Fix	Why
1	Text "not available" / "not applicable" → NULL	Missing values were written as text (for example group_name of knockout matches, given_name of one-name players such as Pelé, unknown birth dates, missing Wikipedia links).
2	squads.shirt_number 0 → NULL	0 means "unknown" (1930–1950 squads). The DDL allows only shirt numbers 1–99.
3	Look up matches.stage_number	The source matches has only the stage name, but group names repeat across stages and groups uses different stage names, so the schema links matches to stages and groups by stage_number.
4	Align matches.group_name with groups	Two source inconsistencies. (a) In the 1982 second stage matches says "Group 1"–"Group 4" where groups says "Group A"–"Group D". (b) The six 1950 final-round matches have no group name, although groups records that round as "Group 1". The code matches each group by its teams and accepts only unique matches; nothing is assumed.
5	Shoot-out scores → NULL when there was no shoot-out	The source stores 0 for the 870 matches without a shoot-out, which would look like a real 0-0 shoot-out. A CHECK in the DDL requires these to be NULL.
6	Keep only the normalised columns	Copied attributes (names, codes, stadium location, ...), same-row derived values (score text, result labels, points, ...) and row numbers (key_id) are not stored (Milestone 2, Section 3.2).
7	Convert types	0/1 → BOOLEAN, text → DATE / TIME, nullable integers. NULL is passed to the database as None.

```
In [6]: # The code of each cleaning step (from src/clean.py)
for label, fn in clean.CLEANING_STEPS:
    print(f"# {label}")
    print(inspect.getsource(fn))
```

```

# 1. Text markers -> NULL
def step_sentinels(name, df, raw):
    """Text markers 'not available' / 'not applicable' -> NULL (cells changed)."""
    n = 0
    for c in df.columns:
        if df[c].dtype == object or pd.api.types.is_string_dtype(df[c]):
            hit = df[c].isin(SENTINELS)
            n += int(hit.sum())
            df[c] = df[c].mask(hit)
    return df, n

# 2. Shirt number 0 -> NULL
def step_shirt_zero(name, df, raw):
    """Squads.shirt_number 0 (unknown) -> NULL (cells changed)."""
    if name != "squads":
        return df, 0
    hit = df["shirt_number"] == 0
    df["shirt_number"] = df["shirt_number"].mask(hit)
    return df, int(hit.sum())

# 3. Look up matches.stage_number
def step_stage_number(name, df, raw):
    """matches: look up stage_number from tournament_stages by (tournament_id, stage_name) (rows mapped)."""
    if name != "matches":
        return df, 0
    stages = raw["tournament_stages"][["tournament_id", "stage_name", "stage_number"]]
    df = df.merge(stages, on=["tournament_id", "stage_name"], how="left", validate="many_to_one")
    assert df["stage_number"].notna().all(), "every match must map to a stage"
    return df, len(df)

# 4. Align group names with groups
def step_group_names(name, df, raw):
    """matches: align group names with groups (1982 second stage, 1950 final round) (rows changed)."""
    if name != "matches":
        return df, 0
    fixes = group_name_fixes(raw)
    df["group_name"] = [fixes[mid][1] if mid in fixes else g for mid, g in zip(df.match_id, df.group_name)]
    return df, len(fixes)

# 5. No shoot-out -> NULL shoot-out scores
def step_shootout_nulls(name, df, raw):
    """Shoot-out scores of 0 -> NULL when there was no shoot-out (cells changed)."""
    cols = {"matches": ["home_team_score_penalties", "away_team_score_penalties"],
            "team_appearances": ["penalties_for", "penalties_against"]}.get(name)
    if not cols:
        return df, 0
    no_so = df["penalty_shootout"] == 0
    for c in cols:
        df[c] = df[c].mask(no_so)
    return df, int(no_so.sum()) * len(cols)

# 6. Keep normalised columns only
def step_select_columns(name, df, raw):
    """Keep only the columns of the normalised table (columns dropped)."""
    n = df.shape[1] - len(COLUMNS[name])
    return df[COLUMNS[name]].copy(), n

# 7. Convert types (bool/date/time/int)
def step_types(name, df, raw):
    """0/1 -> BOOLEAN, text -> DATE / TIME, nullable integers; NULL as None (columns converted)."""
    n = 0
    for c in df.columns:
        if c in BOOL_COLUMNS:
            assert set(df[c].unique()) <= {0, 1}, f"{name}.{c} is not 0/1"
            df[c] = df[c].astype(bool)
            n += 1
        elif c in DATE_COLUMNS:
            df[c] = _to_date(df[c])
            n += 1
        elif c == "match_time":
            df[c] = df[c].map(lambda t: dt.datetime.strptime(t, "%H:%M").time())
            n += 1
        elif c in NULLABLE_INT_COLUMNS or c == "stage_number":
            df[c] = df[c].astype("Int64")
            n += 1
    for c in df.columns:
        # text columns: plain Python objects with None for NULL
        if pd.api.types.is_string_dtype(df[c]) and c not in DATE_COLUMNS:
            df[c] = df[c].astype(object).where(df[c].notna(), None)
    return df, n

```

```
In [7]: cleaning_log = []
cleaned = clean.clean_all(raw, log=cleaning_log)

log = pd.DataFrame(cleaning_log, columns=["table", "step", "changed"])
print("Values / rows / columns changed by each step (all tables):")
print(log.groupby("step")["changed"].sum().to_string())
print()
print("Details per table (columns = step numbers above):")
detail = log.assign(step=log.step.str.split(".").str[0]).pivot_table(index="table",
columns="step", values="changed", aggfunc="sum")
print(detail.fillna(0).astype(int).reindex([t for t in clean.LOAD_ORDER if t in
set(log.table)]).to_string())
```

Values / rows / columns changed by each step (all tables):

step	
1. Text markers -> NULL	11891
2. Shirt number 0 -> NULL	1188
3. Look up matches.stage_number	900
4. Align group names with groups	18
5. No shoot-out -> NULL shoot-out scores	5220
6. Keep normalised columns only	243
7. Convert types (bool/date/time/int)	31

Details per table (columns = step numbers above):

step	1	2	3	4	5	6	7
table							
confederations	0	0	0	0	0	1	0
awards	0	0	0	0	0	1	0
stadiums	0	0	0	0	0	1	0
teams	0	0	0	0	0	3	0
players	455	0	0	0	0	7	1
managers	5	0	0	0	0	1	0
referees	87	0	0	0	0	3	0
tournaments	0	0	0	0	0	13	2
tournament_stages	0	0	0	0	0	7	3
groups	0	0	0	0	0	4	1
qualified_teams	0	0	0	0	0	5	0
host_countries	0	0	0	0	0	5	0
tournament_standings	0	0	0	0	0	4	0
group_standings	0	0	0	0	0	8	2
squads	508	1188	0	0	0	7	1
manager_appointments	5	0	0	0	0	7	0
referee_appointments	0	0	0	0	0	8	0
matches	236	0	900	18	1740	21	9
team_appearances	472	0	0	0	3480	27	3
referee_appearances	236	0	0	0	0	12	0
manager_appearances	505	0	0	0	0	13	0
player_appearances	5334	0	0	0	0	15	2
goals	1021	0	0	0	0	17	2
bookings	812	0	0	0	0	16	3
substitutions	1891	0	0	0	0	16	1
penalty_kicks	303	0	0	0	0	13	1
award_winners	21	0	0	0	0	8	0

```
In [8]: # Group-name fix (step 4), computed from the teams in each match
fixes = clean.group_name_fixes(raw)
fx = pd.DataFrame([(mid, old if old else "(none: 'not applicable')", new) for mid,
(old, new) in fixes.items()],
                  columns=["match_id", "group_name in matches.csv", "group_name
loaded"])
print(fx.groupby(["group_name in matches.csv", "group_name
loaded"]).match_id.agg(["count", lambda s: ", ".join(s)])
      .rename(columns={"<lambda_0>": "matches"}).to_string())
```

	count	matches
group_name in matches.csv group_name loaded		
(none: 'not applicable') Group 1	6	M-1950-17, M-1950-18, M-1950-19, M-1950-20, M-1950-21, M-1950-22
Group 1 Group A	3	M-1982-38, M-1982-42, M-1982-46
Group 2 Group B	3	M-1982-40, M-1982-44, M-1982-48
Group 3 Group C	3	M-1982-39, M-1982-43, M-1982-47
Group 4 Group D	3	M-1982-37, M-1982-41, M-1982-45

```
In [9]: # NULLs after cleaning, per table (only columns that have any)
nulls = [(t, c, int(df[c].isna().sum())) for t, df in cleaned.items() for c in
df.columns if df[c].isna().any()]
print(pd.DataFrame(nulls, columns=["table", "column",
"null_values"]).to_string(index=False))
```

table	column	null_values
players	given_name	371
players	birth_date	78
players	player_wikipedia_link	7
managers	given_name	5
referees	referee_wikipedia_link	87
squads	shirt_number	1188
matches	group_name	230
matches	home_team_score_penalties	870
matches	away_team_score_penalties	870
team_appearances	penalties_for	1740
team_appearances	penalties_against	1740

Proof that nothing is lost. The columns dropped in fix 6 can each be recomputed exactly from the columns we keep. The 33 checks below run on the raw CSVs, and each must say `True`.

```
In [10]: proof = pd.DataFrame(clean.verify_dropped_columns(raw), columns=["check", "passed"])
print(proof.to_string(index=False))
assert proof.passed.all(), "a dropped column is not redundant"
print(f"\n{proof.passed.sum()} / {len(proof)} checks passed")
```

	check	passed
matches.margins = score difference		True
matches.result / win flags from scores and shoot-out scores		True
matches.score text = scores		True
matches.group_stage/knockout_stage depend only on the stage		True
tournament_stages.flags depend only on stage_name		True
team_appearances = matches seen from each side		True
goals.minute_label and match_period from minutes		True
bookings.minute_label and match_period from minutes		True
substitutions.minute_label and match_period from minutes		True
substitutions.going_off = NOT coming_on		True
player_appearances.substitute = NOT starter		True
bookings.sending_off = red_card OR second_yellow_card		True
group_standings.played/goal_difference = W+D+L / GF-GA		True
group_standings.points from wins/draws (2 per win to 1990, 3 from 1994)		True
goals.shirt_number agrees with squads (0 = unknown)		True
bookings.shirt_number agrees with squads (0 = unknown)		True
substitutions.shirt_number agrees with squads (0 = unknown)		True
penalty_kicks.shirt_number agrees with squads (0 = unknown)		True
player_appearances.shirt_number agrees with squads (0 = unknown)		True
award_winners.shared = more than one winner		True
players.position.flags = positions in squads		True
players.count_tournaments = squads per player		True
qualified_teams.count_matches = matches played		True
groups.count_teams = teams in group_standings		True
tournaments.count_teams = qualified teams		True
tournament_stages.count_matches / count_replays from matches		True
tournament_stages.unbalanced_groups from group sizes		True
tournaments.format.flags from tournament_stages		True
tournaments.winner / host_won from standings and hosts		True
referee_appointments.person.attributes = master table		True
referee_appearances.person.attributes = master table		True
manager_appointments.person.attributes = master table		True
manager_appearances.person.attributes = master table		True

33 / 33 checks passed

```
In [11]: # Cleaned shape of each table (the columns match sql/ddl.sql exactly)
for t, df in cleaned.items():
    print(f"{t} ({df.shape[0]} rows, {df.shape[1]} columns): {'',
'.join(df.columns)}")
```

confederations (6 rows, 4 columns): confederation_id, confederation_name, confederation_code, confederation_wikipedia_link
 awards (8 rows, 4 columns): award_id, award_name, award_description, year_introduced
 stadiums (185 rows, 7 columns): stadium_id, stadium_name, city_name, country_name, stadium_capacity, stadium_wikipedia_link, city_wikipedia_link
 teams (84 rows, 8 columns): team_id, team_name, team_code, federation_name, region_name, confederation_id, team_wikipedia_link, federation_wikipedia_link
 players (7907 rows, 5 columns): player_id, family_name, given_name, birth_date, player_wikipedia_link
 managers (357 rows, 5 columns): manager_id, family_name, given_name, country_name, manager_wikipedia_link
 referees (380 rows, 6 columns): referee_id, family_name, given_name, country_name, confederation_id, referee_wikipedia_link
 tournaments (21 rows, 5 columns): tournament_id, tournament_name, year, start_date, end_date
 tournament_stages (107 rows, 9 columns): tournament_id, stage_number, stage_name, start_date, end_date, count_teams, count_scheduled, count_playoffs, count_walkovers
 groups (117 rows, 3 columns): tournament_id, stage_number, group_name
 qualified_teams (457 rows, 3 columns): tournament_id, team_id, performance
 host_countries (22 rows, 2 columns): tournament_id, team_id
 tournament_standings (84 rows, 3 columns): tournament_id, position, team_id
 group_standings (458 rows, 11 columns): tournament_id, stage_number, group_name, position, team_id, wins, draws, losses, goals_for, goals_against, advanced
 squads (10142 rows, 5 columns): tournament_id, team_id, player_id, shirt_number, position_code
 manager_appointments (469 rows, 3 columns): tournament_id, team_id, manager_id
 referee_appointments (511 rows, 2 columns): tournament_id, referee_id
 matches (900 rows, 17 columns): match_id, tournament_id, stage_number, group_name, replayed, replay, match_date, match_time, stadium_id, home_team_id, away_team_id, home_team_score, away_team_score, extra_time, penalty_shootout, home_team_score_penalties, away_team_score_penalties
 team_appearances (1800 rows, 9 columns): match_id, team_id, tournament_id, opponent_id, home_team, goals_for, goals_against, penalties_for, penalties_against
 referee_appearances (900 rows, 3 columns): match_id, tournament_id, referee_id
 manager_appearances (1842 rows, 4 columns): match_id, team_id, manager_id, tournament_id
 player_appearances (18623 rows, 7 columns): match_id, player_id, tournament_id, team_id, position_code, starter, captain
 goals (2548 rows, 10 columns): goal_id, match_id, tournament_id, team_id, player_id, player_team_id, minute_regulation, minute_stoppage, own_goal, penalty
 bookings (2466 rows, 10 columns): booking_id, match_id, tournament_id, team_id, player_id, minute_regulation, minute_stoppage, yellow_card, red_card, second_yellow_card
 substitutions (6464 rows, 8 columns): substitution_id, match_id, tournament_id, team_id, player_id, minute_regulation, minute_stoppage, coming_on
 penalty_kicks (279 rows, 6 columns): penalty_kick_id, match_id, tournament_id, team_id, player_id, converted
 award_winners (132 rows, 4 columns): tournament_id, award_id, player_id, team_id

4. Load with `to_sql(if_exists="append")` in dependency order

Parents are loaded before their children (for example `tournaments` → `tournament_stages` → `groups` → `matches` → `goals`), so every foreign key finds its parent row.

All 27 tables are loaded inside **one transaction**: if any row breaks a constraint, nothing is committed and the database stays empty. This matters on the shared course server. The deferred foreign key on `team_appearances` (each team's opponent row must exist) is checked at COMMIT.

```
In [12]:
timings = []
t0 = time.time()
with engine.begin() as conn:                                     # single transaction
    for the whole load
        for name in clean.LOAD_ORDER:
            t = time.time()
            cleaned[name].to_sql(name, conn, if_exists="append", index=False,
                                method="multi", chunksize=1000)
            timings.append((name, len(cleaned[name]), round(time.time() - t, 2)))
            print(f"loaded {name:22s} {len(cleaned[name]):6d} rows")
    print(f"\nCommitted. Total load time: {time.time() - t0:.1f} s")
```

loaded confederations	6 rows
loaded awards	8 rows
loaded stadiums	185 rows
loaded teams	84 rows

loaded players	7907 rows
loaded managers	357 rows
loaded referees	380 rows
loaded tournaments	21 rows
loaded tournament_stages	107 rows
loaded groups	117 rows
loaded qualified_teams	457 rows
loaded host_countries	22 rows
loaded tournament_standings	84 rows
loaded group_standings	458 rows
loaded squads	10142 rows
loaded manager_appointments	469 rows
loaded referee_appointments	511 rows
loaded matches	900 rows
loaded team_appearances	1800 rows
loaded referee_appearances	900 rows
loaded manager_appearances	1842 rows
loaded player_appearances	18623 rows
loaded goals	2548 rows
loaded bookings	2466 rows
loaded substitutions	6464 rows
loaded penalty_kicks	279 rows
loaded award_winners	132 rows

Committed. Total load time: 12.3 s

5. Verify row counts against the CSV files

```
In [13]: db_counts = {t: pd.read_sql_query(f"SELECT count(*) AS n FROM {t}", engine).n[0] for
t in clean.LOAD_ORDER}
check = csv_summary[["table", "csv_rows"]].assign(db_rows=lambda d:
d.table.map(db_counts))
check["status"] = (check.csv_rows == check.db_rows).map({True: "OK", False:
"MISMATCH"})
print(check.to_string(index=False))
print(f"\nTotal: {check.csv_rows.sum()} CSV rows, {check.db_rows.sum()} database
rows")
assert (check.status == "OK").all(), "row count mismatch"
print("All 27 tables match their CSV files.")
```

table	csv_rows	db_rows	status
confederations	6	6	OK
awards	8	8	OK
stadiums	185	185	OK
teams	84	84	OK
players	7907	7907	OK
managers	357	357	OK
referees	380	380	OK
tournaments	21	21	OK
tournament_stages	107	107	OK
groups	117	117	OK
qualified_teams	457	457	OK
host_countries	22	22	OK
tournament_standings	84	84	OK
group_standings	458	458	OK
squads	10142	10142	OK
manager_appointments	469	469	OK
referee_appointments	511	511	OK
matches	900	900	OK
team_appearances	1800	1800	OK
referee_appearances	900	900	OK
manager_appearances	1842	1842	OK
player_appearances	18623	18623	OK
goals	2548	2548	OK
bookings	2466	2466	OK
substitutions	6464	6464	OK
penalty_kicks	279	279	OK
award_winners	132	132	OK

Total: 57269 CSV rows, 57269 database rows
All 27 tables match their CSV files.

6. Integrity checks on the loaded database

These checks run in SQL with `pd.read_sql_query` and test things the constraints alone cannot guarantee.

```
In [14]: checks_sql = {
    "team-matches where goal rows differ from the score": """
        SELECT count(*) AS n FROM team_appearances ta
        LEFT JOIN (SELECT match_id, team_id, count(*) AS g FROM goals GROUP BY
match_id, team_id) g
            ON g.match_id = ta.match_id AND g.team_id = ta.team_id
        WHERE COALESCE(g.g, 0) <> ta.goals_for""",
    "matches without exactly 2 team appearances": """
        SELECT count(*) AS n FROM matches m
        WHERE (SELECT count(*) FROM team_appearances ta WHERE ta.match_id =
m.match_id) <> 2""",
    "matches without a referee": """
        SELECT count(*) AS n FROM matches m
        WHERE NOT EXISTS (SELECT 1 FROM referee_appearances r WHERE r.match_id =
m.match_id)""",
    "team_appearances that disagree with matches": """
        SELECT count(*) AS n FROM team_appearances ta JOIN matches m ON m.match_id =
ta.match_id
        WHERE (ta.home_team AND (ta.team_id <> m.home_team_id OR ta.goals_for <>
m.home_team_score))
            OR (NOT ta.home_team AND (ta.team_id <> m.away_team_id OR ta.goals_for <>
m.away_team_score))""",
    "shoot-out kicks that disagree with shoot-out scores": """
        SELECT count(*) AS n FROM team_appearances ta
        WHERE ta.penalties_for IS NOT NULL
            AND ta.penalties_for <> (SELECT count(*) FROM penalty_kicks p
            WHERE p.match_id = ta.match_id AND p.team_id =
ta.team_id AND p.converted)""",
    "group-stage matches with no group / knockout matches with a group": """
        SELECT count(*) AS n FROM matches m JOIN tournament_stages s USING
(tournament_id, stage_number)
        WHERE (s.stage_name IN ('group stage', 'second group stage', 'final round'))
<> (m.group_name IS NOT NULL)""",
    "matches outside their tournament's dates": """
        SELECT count(*) AS n FROM matches m JOIN tournaments t USING (tournament_id)
        WHERE m.match_date NOT BETWEEN t.start_date AND t.end_date""",
}
res = pd.DataFrame([(k, pd.read_sql_query(q, engine).n[0]) for k, q in
checks_sql.items()], columns=["check", "problem_rows"])
print(res.to_string(index=False))
assert (res.problem_rows == 0).all()
print("\nAll integrity checks passed (0 problem rows).")
```

	check	problem_rows
	team-matches where goal rows differ from the score	0
	matches without exactly 2 team appearances	0
	matches without a referee	0
	team_appearances that disagree with matches	0
	shoot-out kicks that disagree with shoot-out scores	0
group-stage matches with no group / knockout matches with a group		0
	matches outside their tournament's dates	0

All integrity checks passed (0 problem rows).

```
In [15]: # Spot checks on the Loaded data
q = """
SELECT t.year, m.match_id, ht.team_name AS home, m.home_team_score AS hs,
m.away_team_score AS as_,
    at.team_name AS away, s.stage_name, m.extra_time, m.penalty_shootout,
```

```

        m.home_team_score_penalties AS hp, m.away_team_score_penalties AS ap
FROM matches m
JOIN tournaments t          ON t.tournament_id = m.tournament_id
JOIN tournament_stages s    ON s.tournament_id = m.tournament_id AND s.stage_number =
m.stage_number
JOIN teams ht               ON ht.team_id = m.home_team_id
JOIN teams at               ON at.team_id = m.away_team_id
WHERE s.stage_name = 'final'
ORDER BY t.year DESC
"""

finals = pd.read_sql_query(q, engine)
print("World Cup finals, most recent first:")
print(finals.head(10).to_string(index=False))

```

World Cup finals, most recent first:

year	match_id	home	hs	as_	away	stage_name	extra_time	penalty_shootout	hp	ap
2018	M-2018-64	France	4	2	Croatia	final	False	False	NaN	NaN
2014	M-2014-64	Germany	1	0	Argentina	final	True	False	NaN	NaN
2010	M-2010-64	Netherlands	0	1	Spain	final	True	False	NaN	NaN
2006	M-2006-64	Italy	1	1	France	final	True	True	5.0	3.0
2002	M-2002-64	Germany	0	2	Brazil	final	False	False	NaN	NaN
1998	M-1998-64	Brazil	0	3	France	final	False	False	NaN	NaN
1994	M-1994-52	Brazil	0	0	Italy	final	True	True	3.0	2.0
1990	M-1990-52	West Germany	1	0	Argentina	final	False	False	NaN	NaN
1986	M-1986-52	Argentina	3	2	West Germany	final	False	False	NaN	NaN
1982	M-1982-52	Italy	3	1	West Germany	final	False	False	NaN	NaN

In [16]:

```

q = """
SELECT player_id, family_name, given_name, birth_date
FROM players WHERE given_name IS NULL OR birth_date IS NULL
ORDER BY player_id
"""

df = pd.read_sql_query(q, engine)
print(f"{len(df)} players with a NULL given name or birth date (were text markers or
empty in the CSV), first 10:")
print(df.head(10).to_string(index=False))

```

445 players with a NULL given name or birth date (were text markers or empty in the CSV), first 10:

player_id	family_name	given_name	birth_date
P-00005	Rapp	Herman	None
P-00008	Rivaldo	NaN	1972-04-19
P-00023	Perácio	NaN	1917-11-02
P-00053	Aguirre	Francisco	None
P-00078	Hilario	NaN	1905-12-08
P-00113	Costinha	NaN	1974-12-01
P-00135	Johnsson	Carl	None
P-00148	Dunga	NaN	1963-10-31
P-00162	Brandão	NaN	1911-04-21
P-00174	Índio	NaN	1931-03-01

7. Summary

- **Tables:** all 28 were created from `sql/ddl.sql` (the 27 dataset tables plus the `positions` lookup).
- **Rows:** all 57,269 CSV rows were loaded with `pd.read_csv` → cleaning → `DataFrame.to_sql(if_exists="append", index=False)`, in dependency order and in one transaction. Every table's row count matches its CSV.
- **Cleaning:** the 7 fixes were applied and counted. Text markers and placeholder values became NULL, 0/1 flags became BOOLEAN, text dates became DATE, `stage_number` was added to matches, and 18 match group names were aligned with `groups` (1982 second stage and 1950 final round). Dropped columns were proven redundant (33/33 checks).
- **Integrity:** all integrity checks report 0 problem rows. For example, the goals table reproduces every match score.

Running on the course server: connect to the university network, put the server details in `.env` , then use **Kernel > Restart Kernel and Run All Cells**. The notebook prints the target (without the password), skips the local-only reset, refuses to load into tables that already hold data, and loads everything in a single transaction. See `docs/university_server_guide.md` .